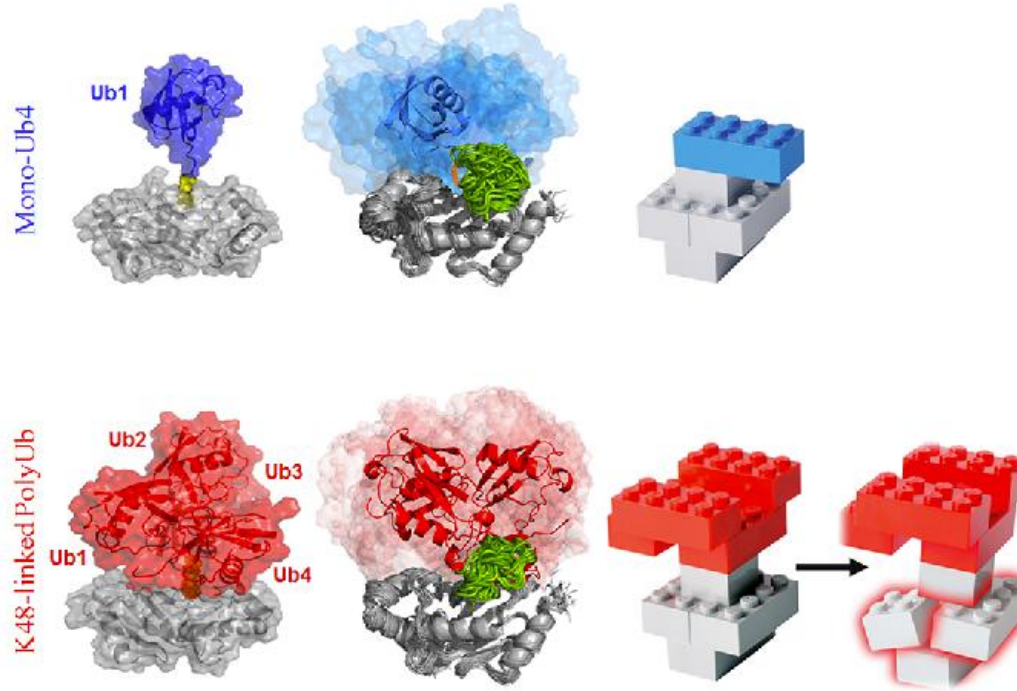


AlphaFold 2:

*How domain-science-informed neural architectures
can matter more than big data, through the lens of protein folding*

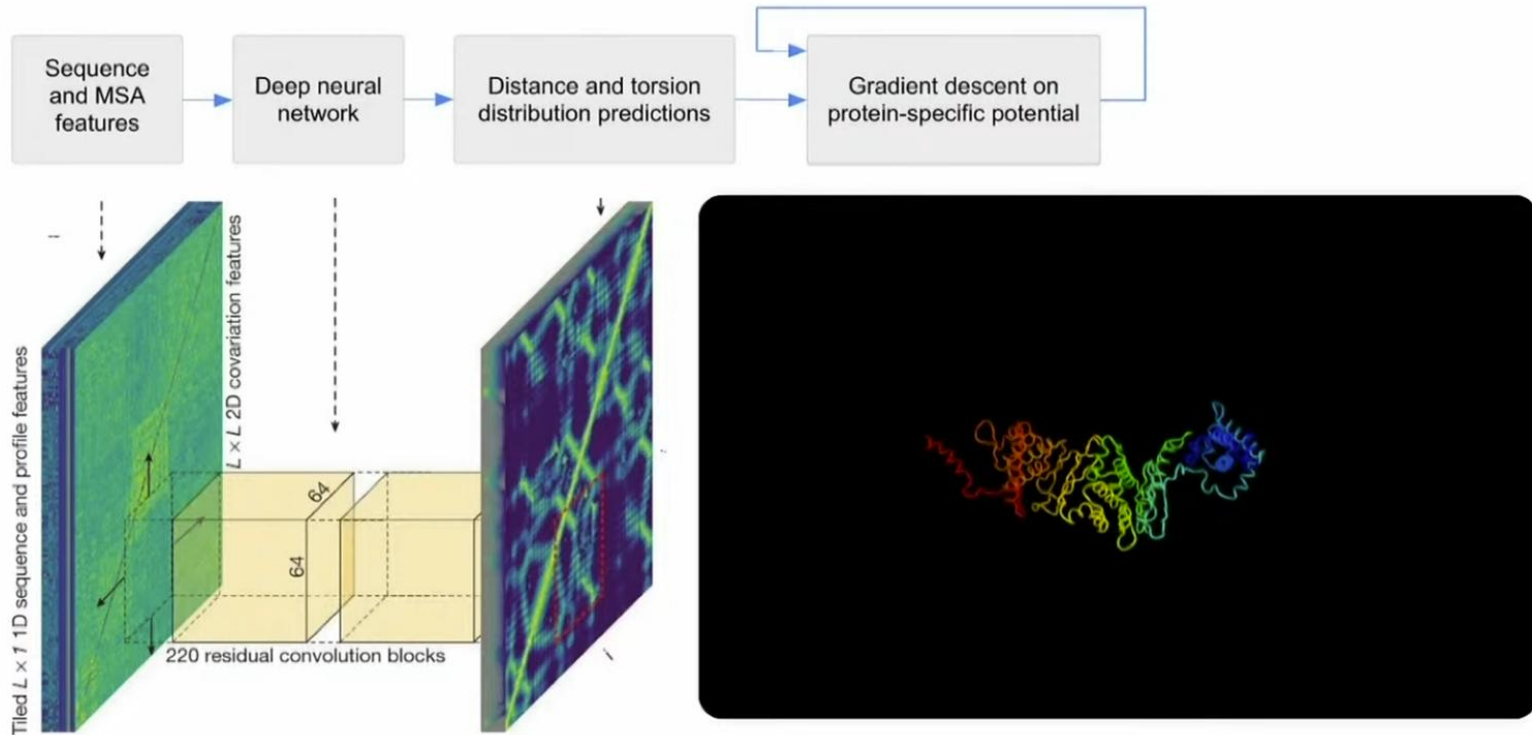
Introduction

Proteins Are Bio-LEGO-Like Structures



“Protein Assembly and Building Blocks: Beyond the Limits of the **LEGO** Brick **Metaphor**.” by Y. Levy

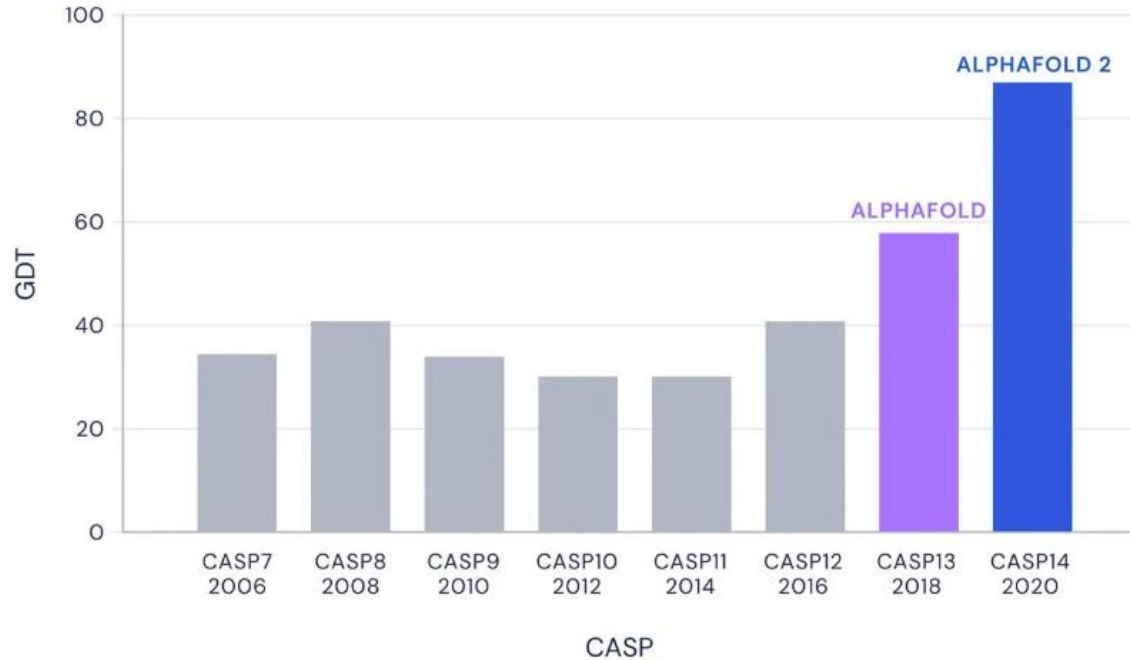
AlphaFold 1 (2018)



AlphaFold 1 was more like a **vision network** applied to distogram maps.

AlphaFold 1 vs AlphaFold 2 Performance

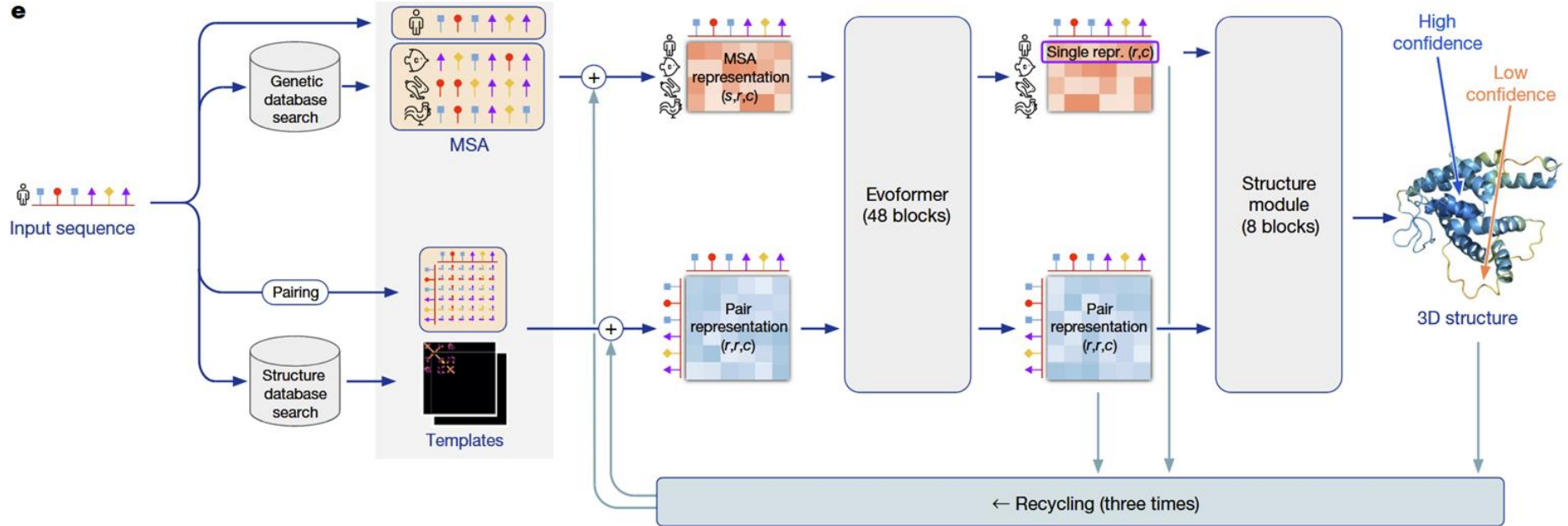
Median Free-Modelling Accuracy



CASP is a **biennial international competition** that evaluates how well computational methods can **predict the 3D structure of proteins** from their amino-acid sequence.

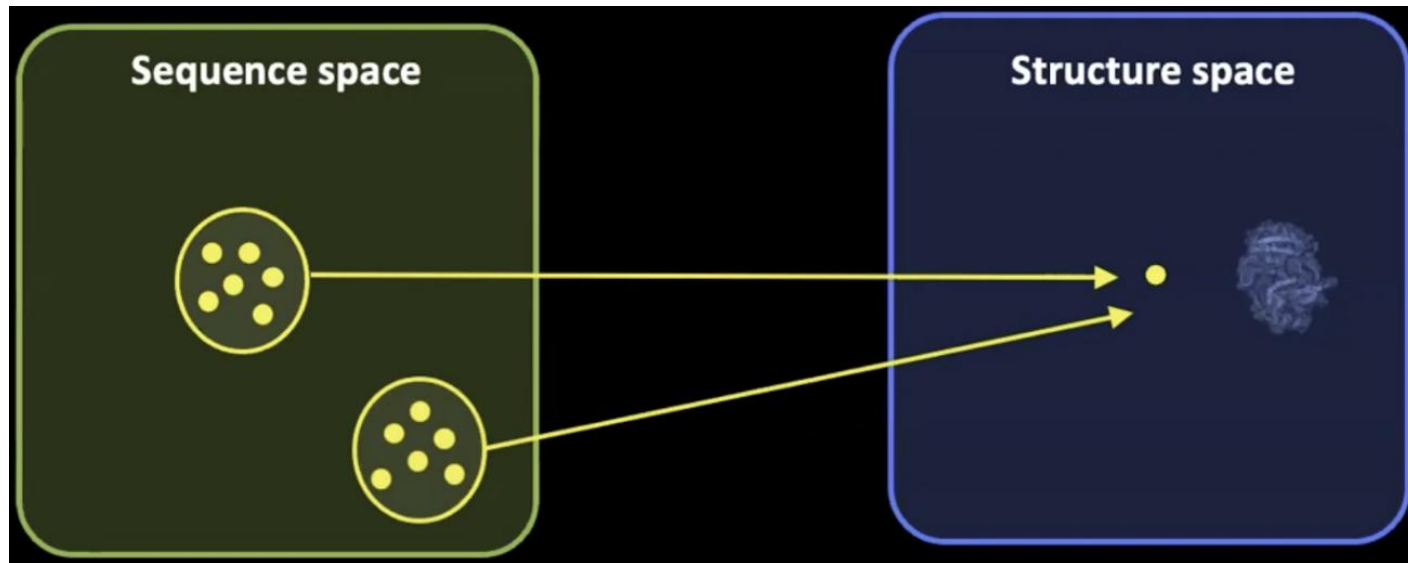
AlphaFold 2

AlphaFold 2 (2020)



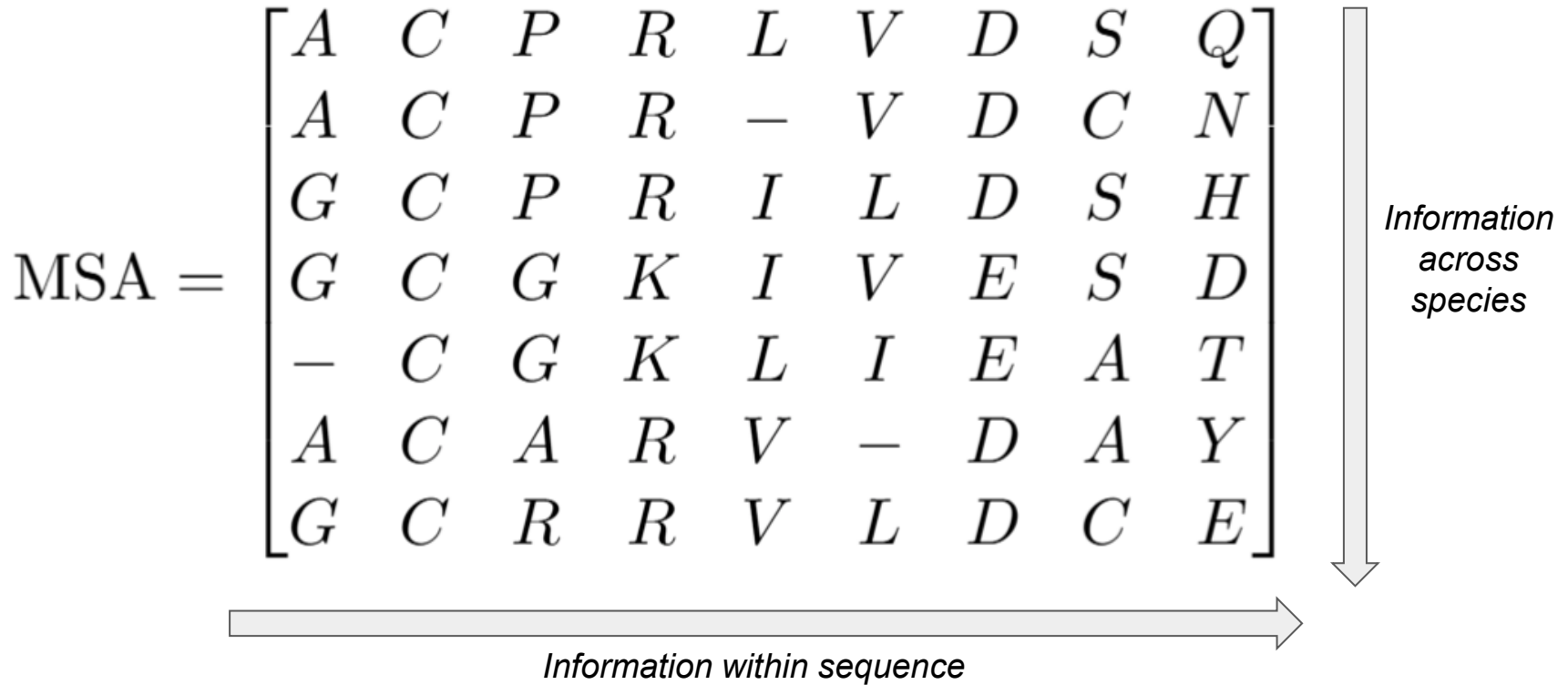
“Highly accurate protein structure prediction with AlphaFold”, by John Jumper et al.

Sequence-Structure Mapping

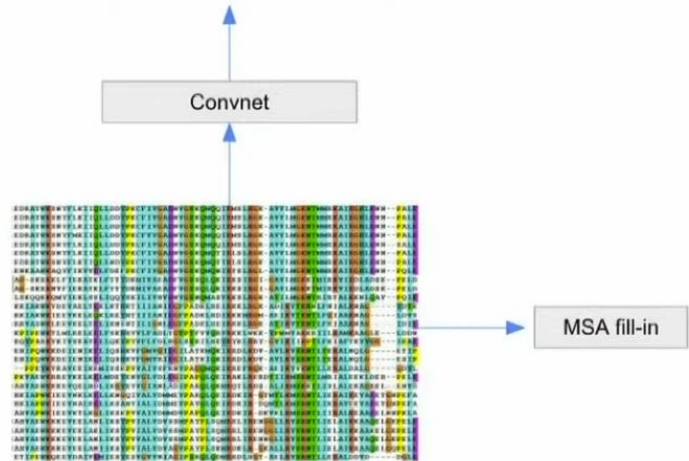
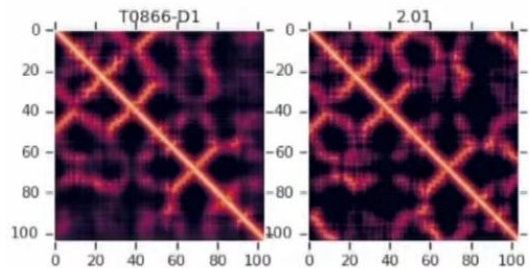


- Many different amino-acid sequences can converge to a similar 3D fold, so **sequence space is much larger** than **structure space**.
- This **many-to-one mapping** can actually **aid generalization**: diverse sequences may share the same overall fold, although predicting fine structural details remains challenging.

Multiple Sequence Alignments (MSA)

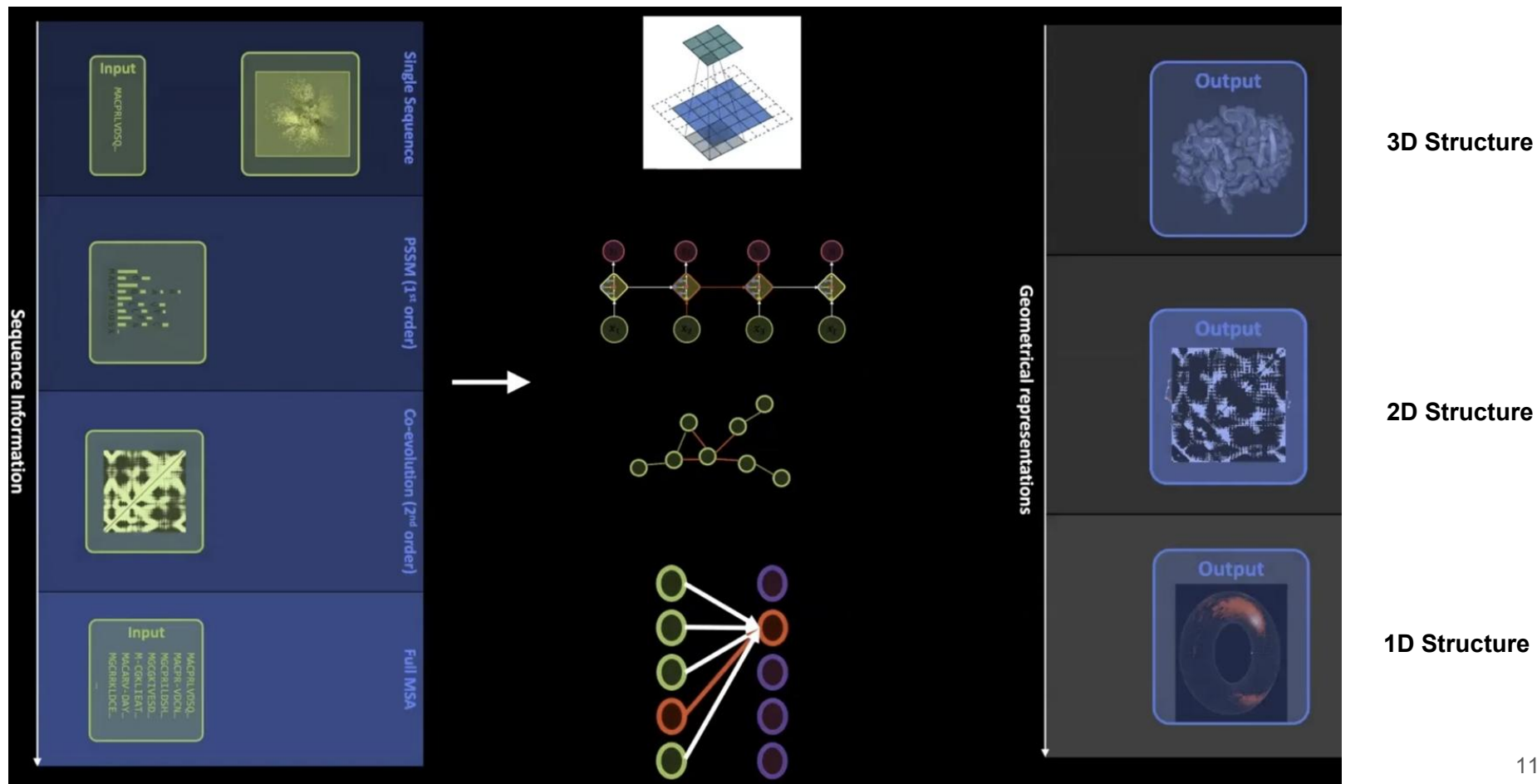


Using Evolutionary Data



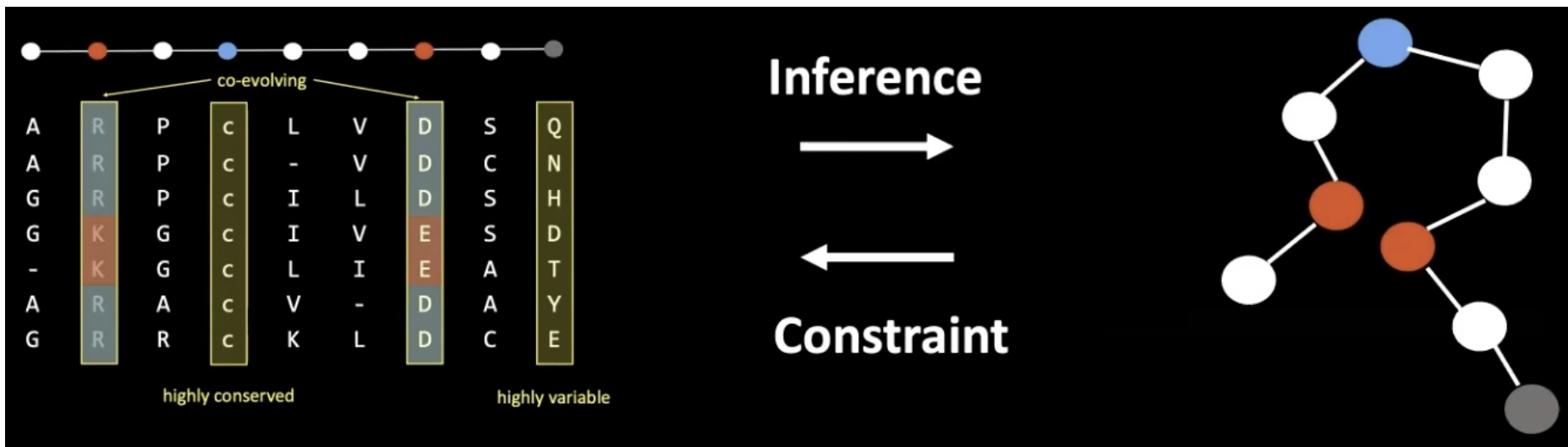
- BERT-style models on MSAs learned to **recover masked evolutionary information**
- **Axial attention alternates between processing within sequences and across sequences**
- This architectural choice **aligned the network operations with the biological structure of the problem**

AlphaFold 2 Links 1D, 2D, and 3D Representations



Modelling Co-Evolution

The **key inductive bias** we want to model is the idea of **co-evolution**

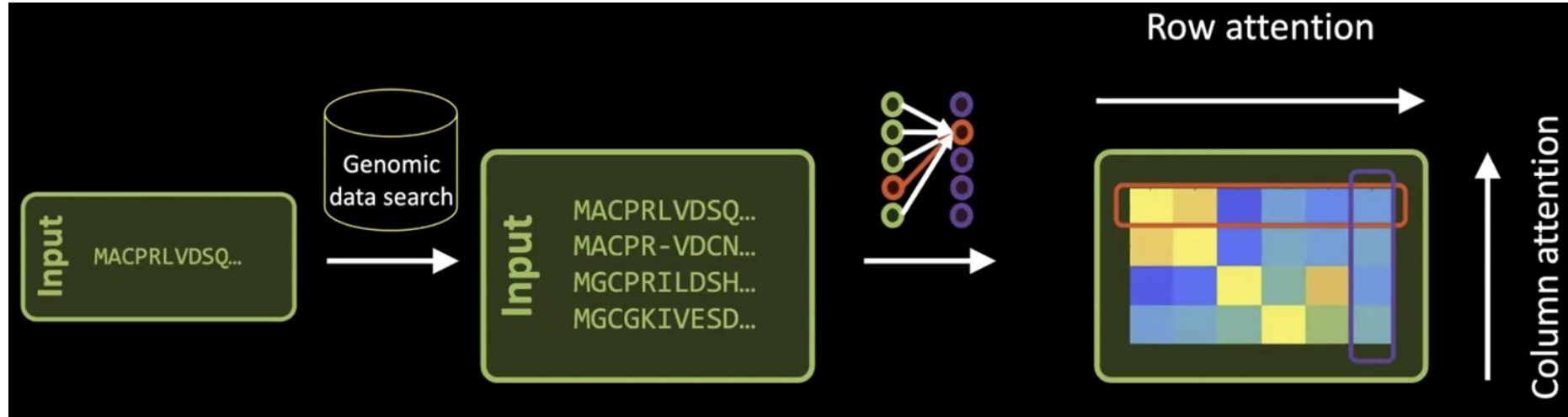


Note that the **red** tokens are the **residues**
(A residue is an amino acid unit within a protein chain)

Row Attention Integrates Information Within a Sequence



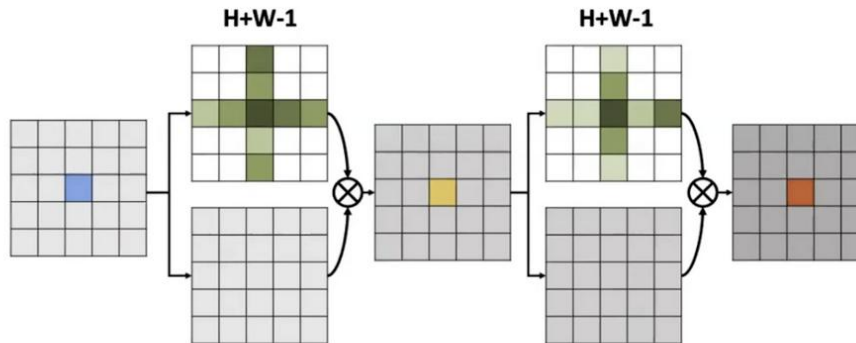
Column Attention Integrates Information Across Sequences



Axial Attention Alternates Between Rows and Columns



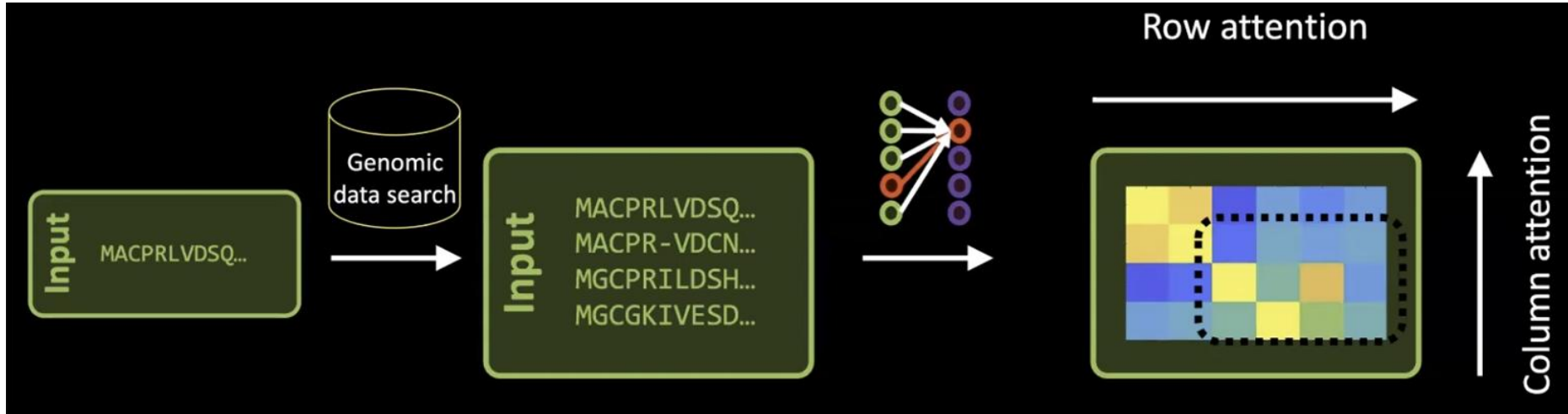
***Criss-cross** attends to the row and column of each position **simultaneously**. **Axial** attention **alternates**: first all **rows**, then all **columns**. Both achieve global coverage, but axial is simpler.*



(b) Criss-Cross Attention block

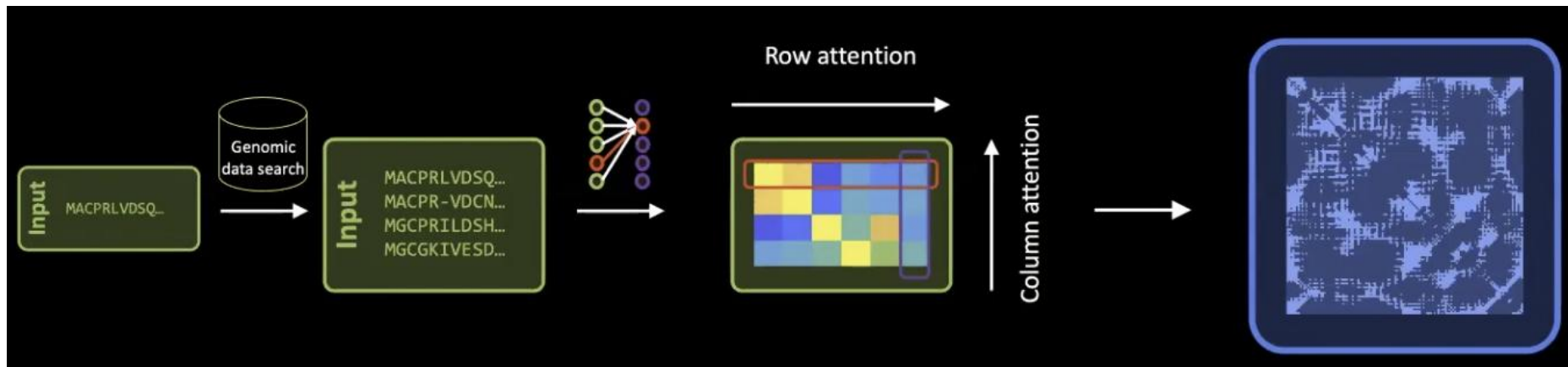
- AlphaFold 2 uses **axial attention** by **alternating row** and **column** attention
- This **separates processing** across sequences from processing along residue positions

Cropping Makes Training Tractable



AlphaFold uses cropping **during training**, but **not at inference** time.

The Evoformer First Predicts Pairwise Distances



First conceptual step is the prediction of a distogram. The **distogram may indicate the geometry** or fold of the protein.

*Note → A **distogram** is a matrix where entry (i, j) represents the predicted distance between residue i and residue j , specifically between their C^α atoms (the central carbon of each residue's backbone). So for a protein with L residues, you get an $L \times L$ matrix.*

MSA Transformer Helped Establish the Design Direction



bioRxiv
THE PREPRINT SERVER FOR BIOLOGY

MSA Transformer

Roshan Rao, Jason Liu, Robert Verkuil, Joshua Meier, John F. Canny, Pieter Abbeel, Tom Sercu, Alexander Rives

doi: <https://doi.org/10.1101/2021.02.12.430858>

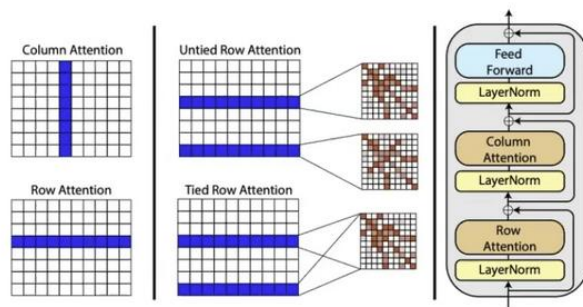
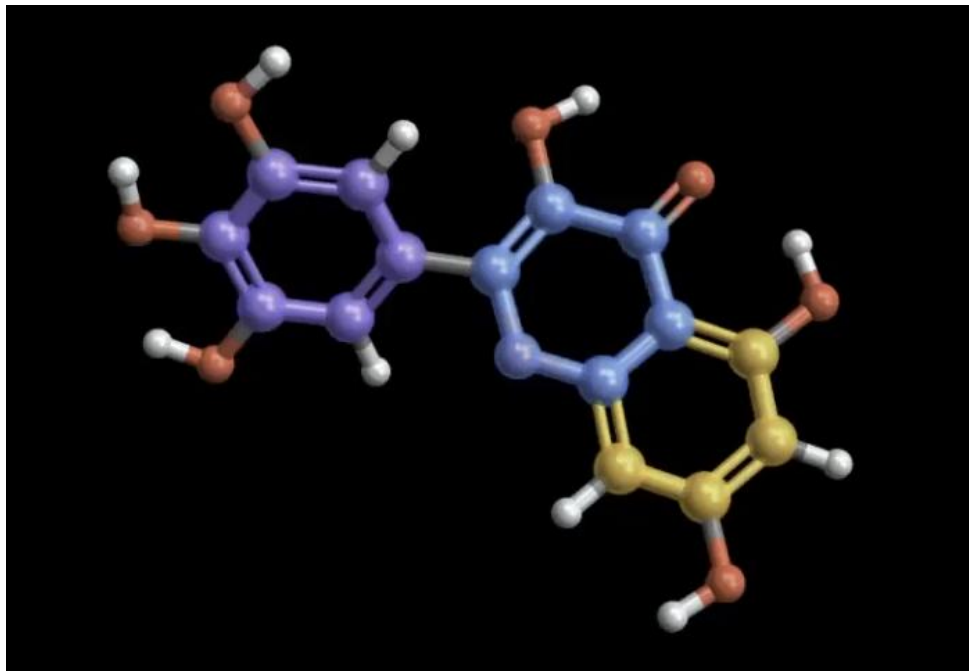


Table 3. Supervised contact prediction on CASP13 and CAMEO (long-range precision). *Uses outer-concatenation of the query sequence representation as features. †Additionally uses the row attention maps as features.

Model	CASP13-FM		CAMEO	
	L	L/5	L	L/5
trRosetta _{base}	45.7	69.6	50.9	75.5
trRosetta _{full}	51.8	80.1	53.2	77.5
Co-evolutionary	40.1	65.2	47.3	72.1
ProTrans-T5	25.0	41.4	40.8	63.3
ESM-1b	28.2	50.2	44.4	68.4
MSA Transformer*	54.5	80.2	53.6	78.0
MSA Transformer†	54.6	77.5	55.8	79.1

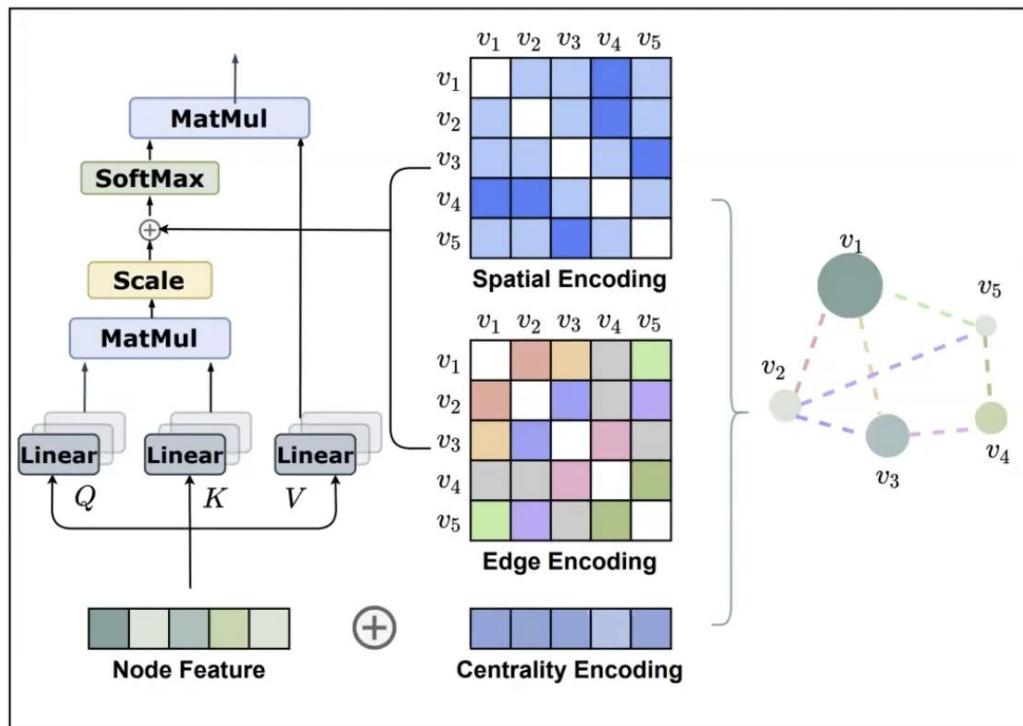
Proteins “are” Graphs

A chemical molecule can be represented as a graph, so we need to **encode relational structure**



Standard Transformers do not model edge information explicitly

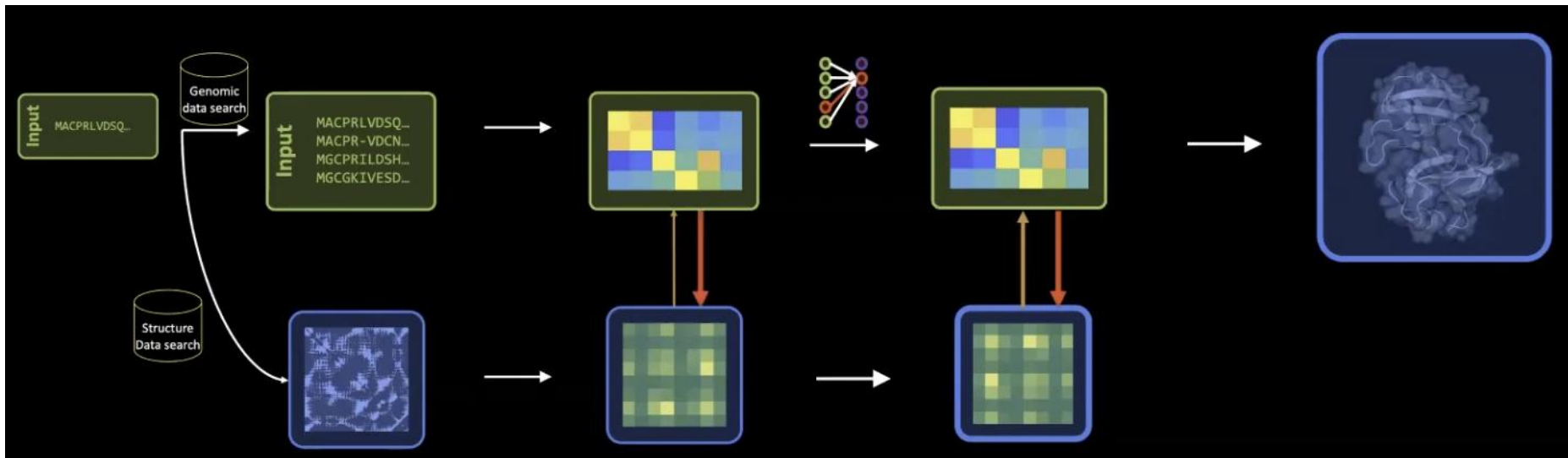
Transformer Graph Representations



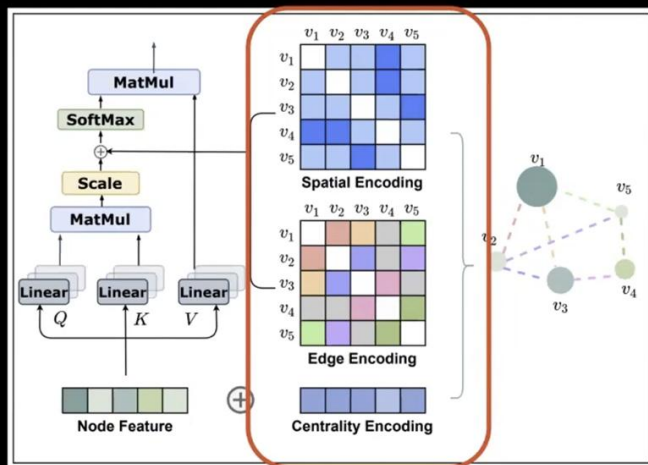
Extending Transformers to encode Graph Edges

“Do Transformers Really Perform Bad for Graph Representation?” by C. Ying et al., NeurIPS 2021

MSA and Pair Representations Exchange Information



Attention Bias Can Encode Pairwise Edge Information

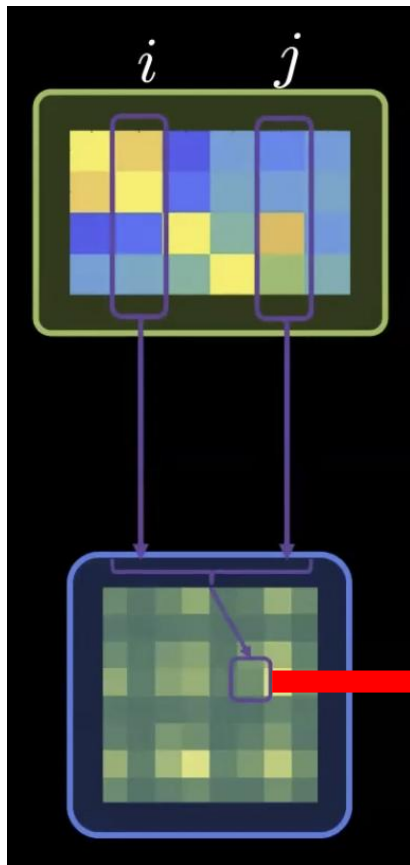


$$a_{ij} = \text{softmax}(q_i^T k_j + b_{ij})$$

Bias b == Edge Information

C. Ying et al., Do Transformers Really Perform Badly for Graph Representation? NeurIPS 2021

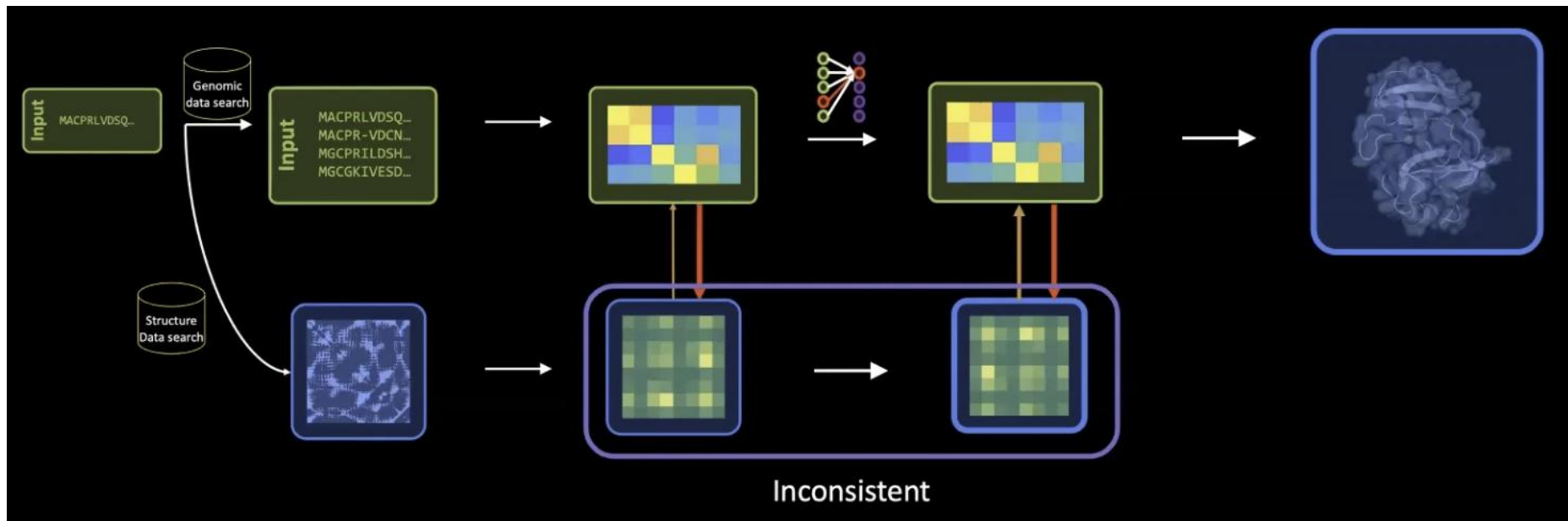
The Pair Representation Is Updated from the MSA



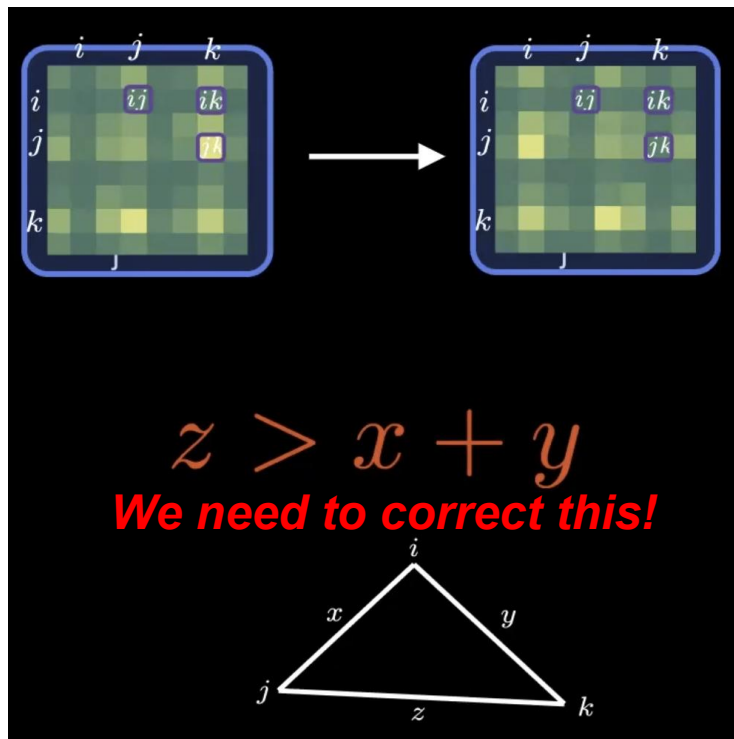
$$O_{ij} = \text{mean} (a_i \otimes b_j)$$

Take the **mean** of the **outer product** of 2 columns of the MSA.

Pairwise Predictions Must Be Geometrically Consistent



Triangle Constraints Enforce Geometric Consistency

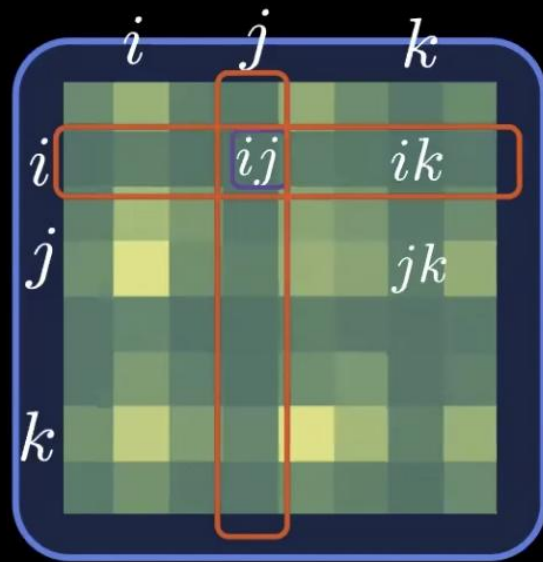


We must **obey** the triangle inequality $\rightarrow z \leq x + y$

AlphaFold 2 Enforces Triangle Constraints Through Smooth Updates

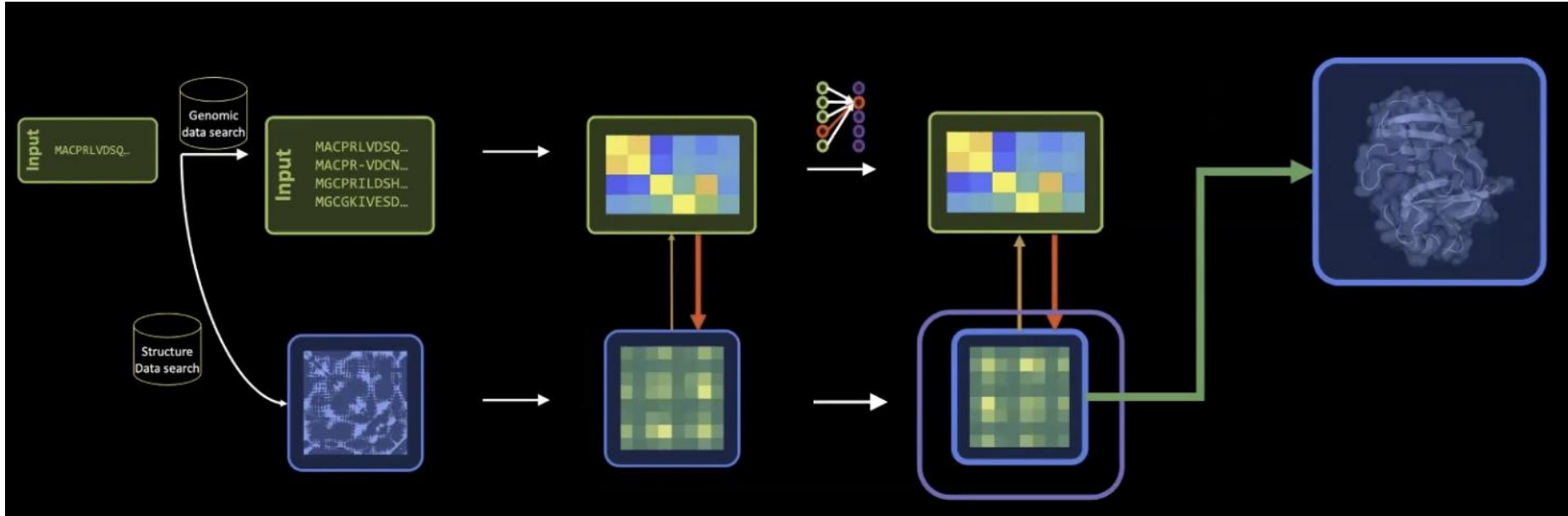
Expensive!
But worth it.

$$z_{ij} \leftarrow f\left(\sum_k a_{ik} b_{jk}\right)$$



Triangle consistency is encouraged through **smooth** updates of the 2D pair representation

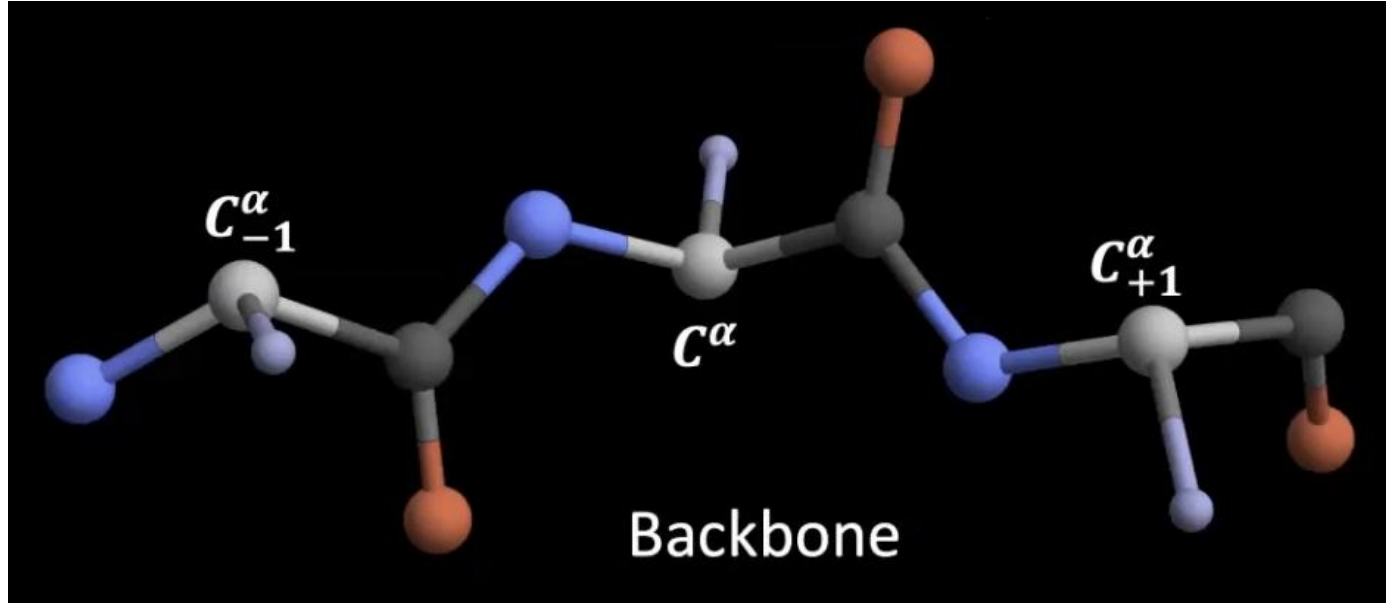
The Structure Module Converts Representations into 3D Coordinates



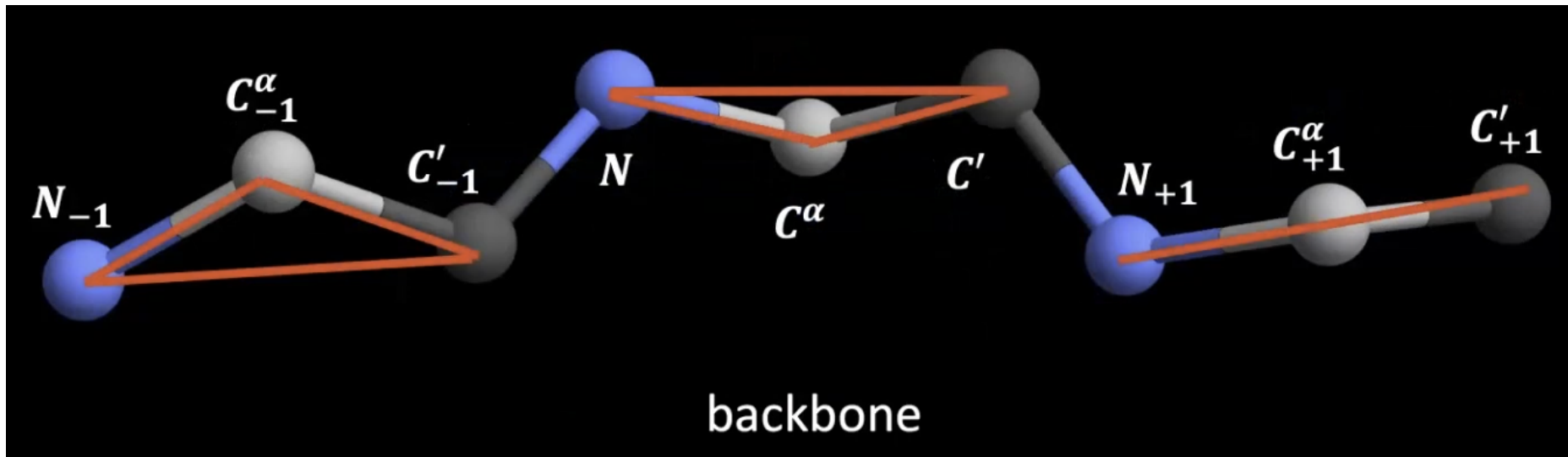
To produce 3D structure, AlphaFold uses the **Pair Representation** together with **the First Row of the MSA**, corresponding to the target sequence.

There are **many more pairwise relationships than there are coordinates to find**, so the structure is heavily constrained → i.e. the model has more than enough information to pin down the 3D shape.

The Protein Backbone Provides the Structural Scaffold

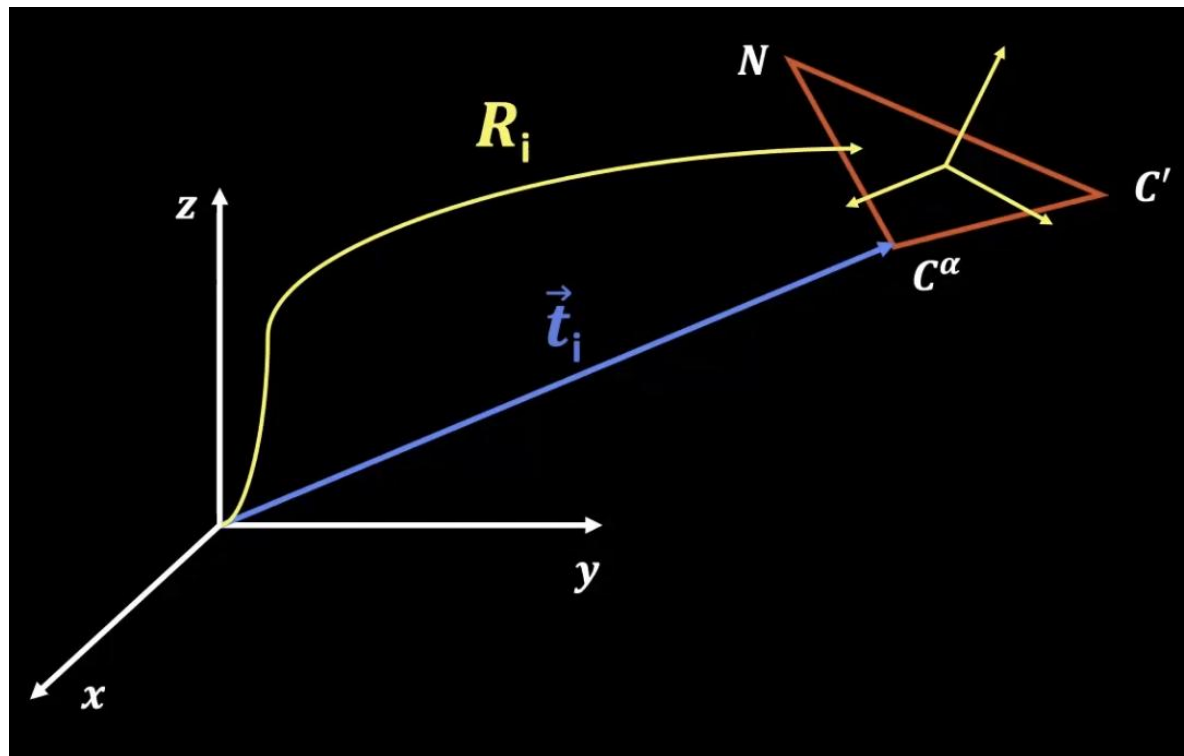


Each Residue Is Represented as a Rigid Local Frame



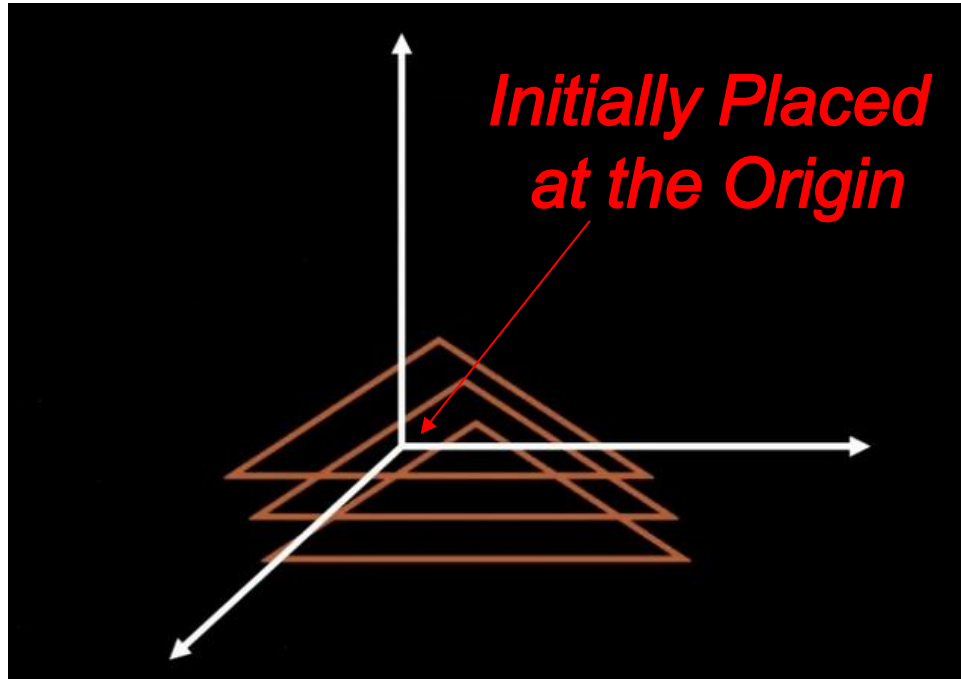
- Each residue is represented by a rigid **Local Frame** (red triangles).
- Structure prediction requires learning the **position and orientation** of these frames in **Euclidean Space**.

Structure Prediction Becomes Rotation and Translation of Local Frames



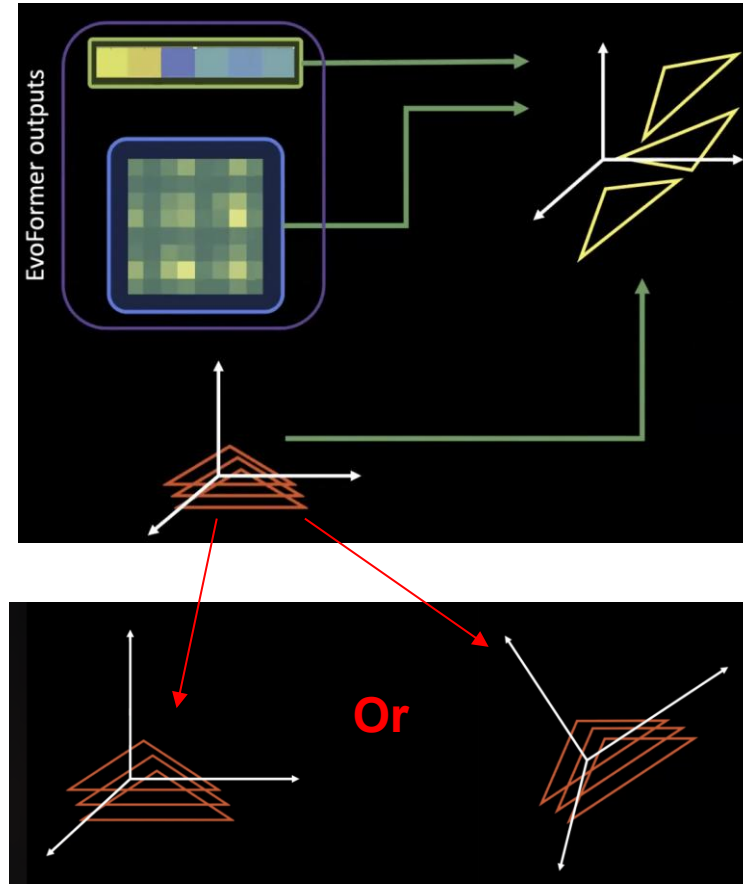
Our task is to learn the **rotation matrix R** and **translation vector t** for each of the triangles. There are 2 frames: **global coordinates frame** (white) and **local coordinates frame** (yellow).

“Black hole” Initial Conditions

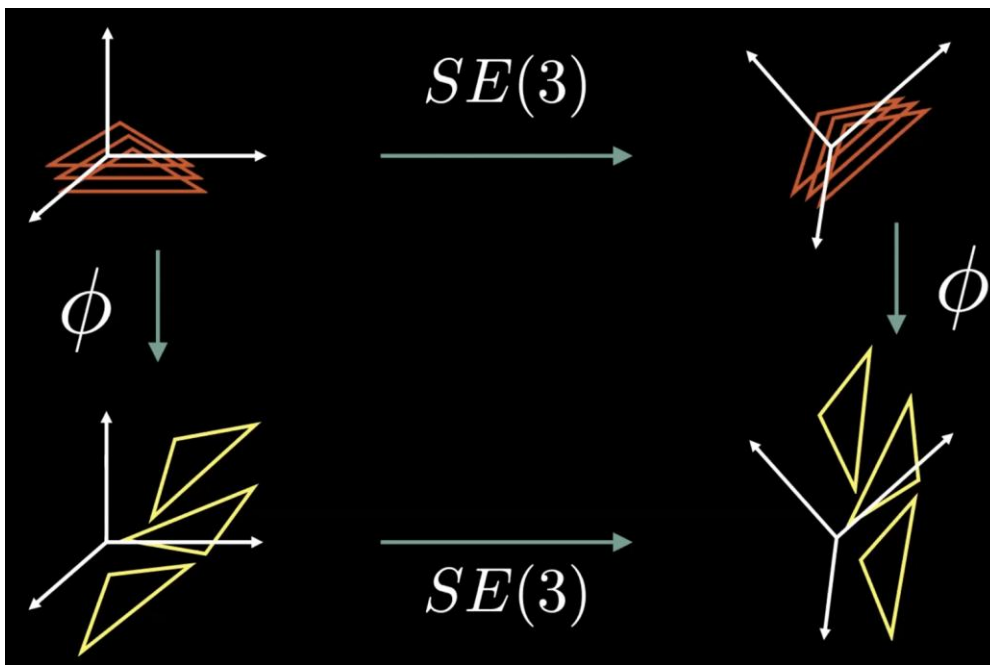


Initial conditions → **collapse all the triangles** (i.e. bag of words) **at the origin**, such that the $\mathbf{C}^\alpha$ atom is sitting at the origin of the global reference frame, and all the local reference frames are oriented in the same direction as the global reference frame.

Problem Is: There Is No Meaningful Absolute Coordinate System



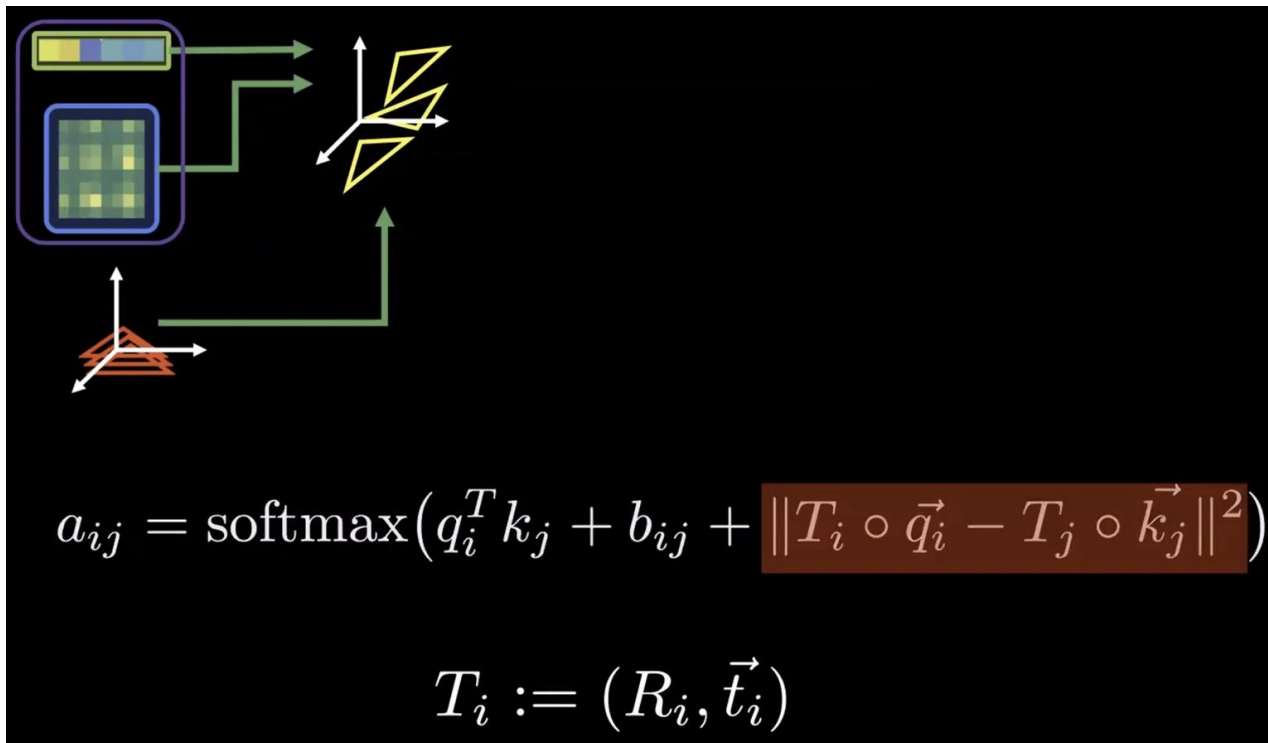
The Structure Module Must Be Rotation-Equivariant



The “**Evoformer**” operates on abstract feature representations with no geometric meaning. But the “**Structure Module**” must produce actual 3D coordinates. And since a protein has no preferred orientation in space, the structure module must be **SE(3)-equivariant**: *rotating the input must rotate the output by the same amount*. So both paths around the diagram must agree.

Invariant Point Attention Injects Geometry into Attention

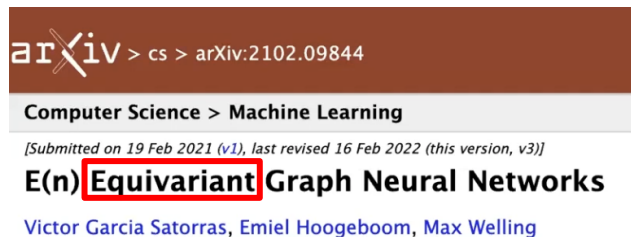
The geometrically aware attention formula becomes



This attention is called the “**Invariant Point Attention**”. It was novel at the time. We allow the queries and keys to live in the Euclidean space.

Invariant Point Attention Was Inspired by Equivariant Models

The type of equivariance we are using in AlphaFold 2 is **approximate**.

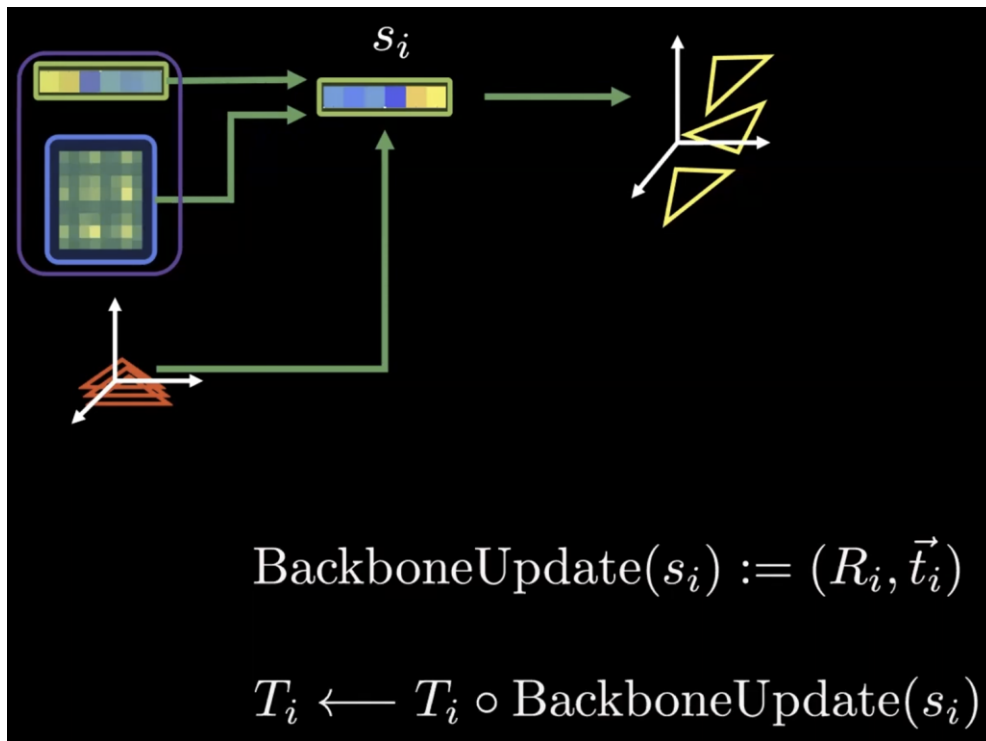


$$\mathbf{x}_i^{l+1} = \mathbf{x}_i^l + C \sum_{j \neq i} (\mathbf{x}_i^l - \mathbf{x}_j^l) \phi_x(\mathbf{m}_{ij})$$

$$T_i^{l+1} \leftarrow T_i^l \circ \text{BackboneUpdate}(s_i)$$

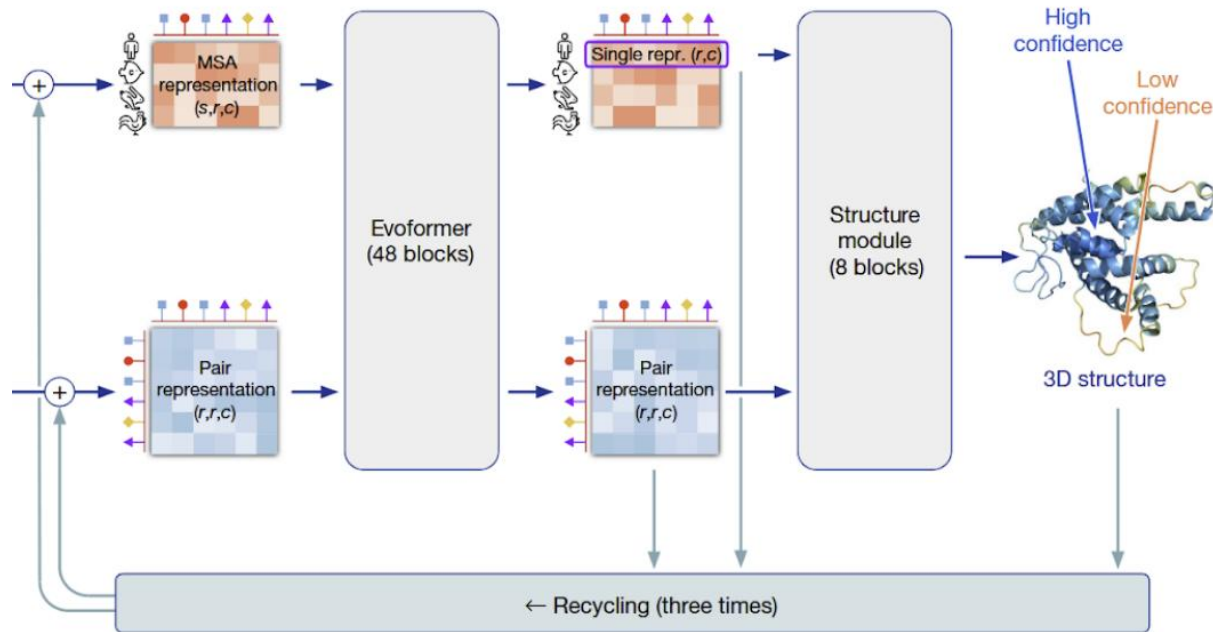
Note that we need not necessarily use sophisticated and mathematically principled techniques from **group theory** and **differential geometry**.

BackboneUpdate Predicts Local Frame Transformations



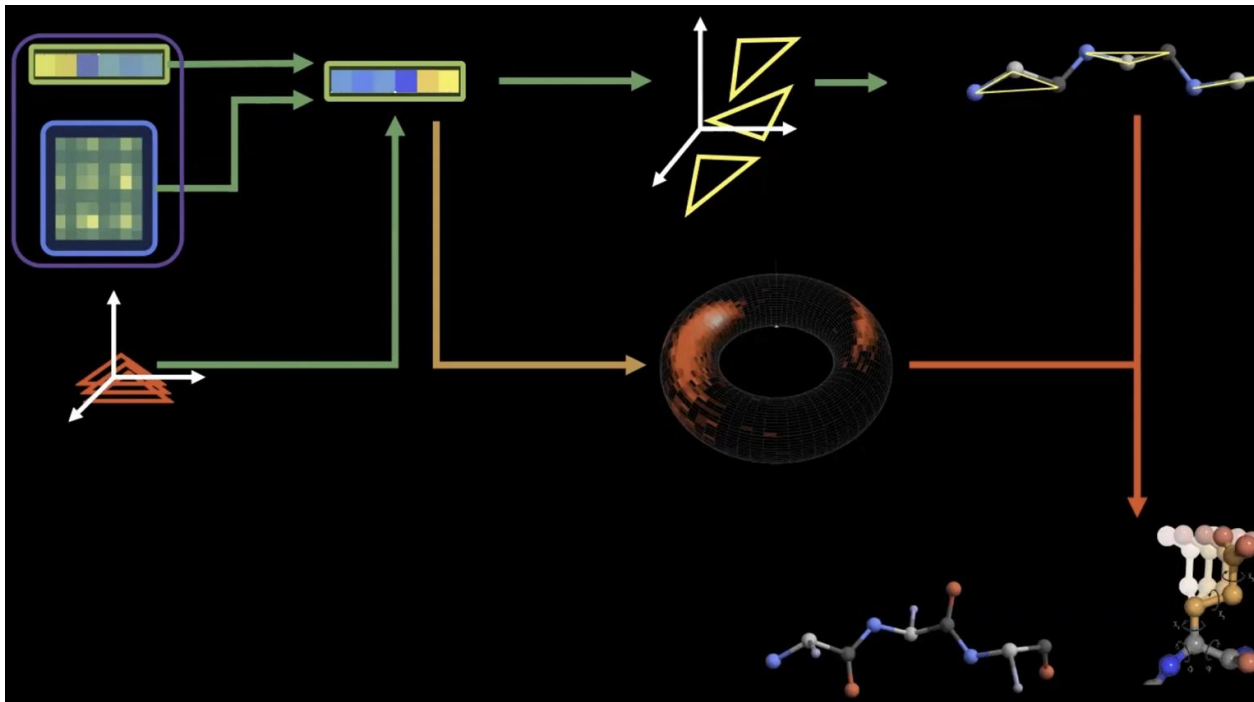
The updated single representation $\mathbf{S}_i$ is passed to **BackboneUpdate()**, which predicts the local **rotation** and **translation for each residue frame**.

Recycling Iteratively Refines the Predicted Structure



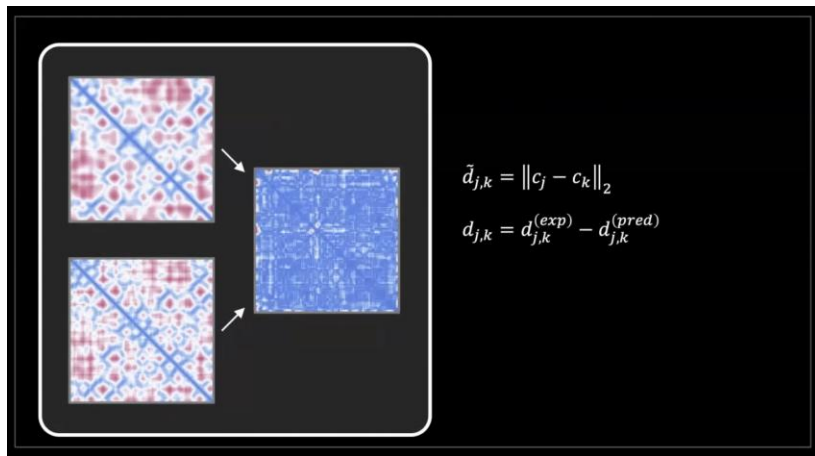
- There are **48 Evoformer blocks**
- Followed by **8 Structure module blocks**
- **Loss** is applied **after each block**
- **Recycling** $\approx$ iterative refinement (3 cycles)

Side-chains Are Predicted from the Same Structural Representation

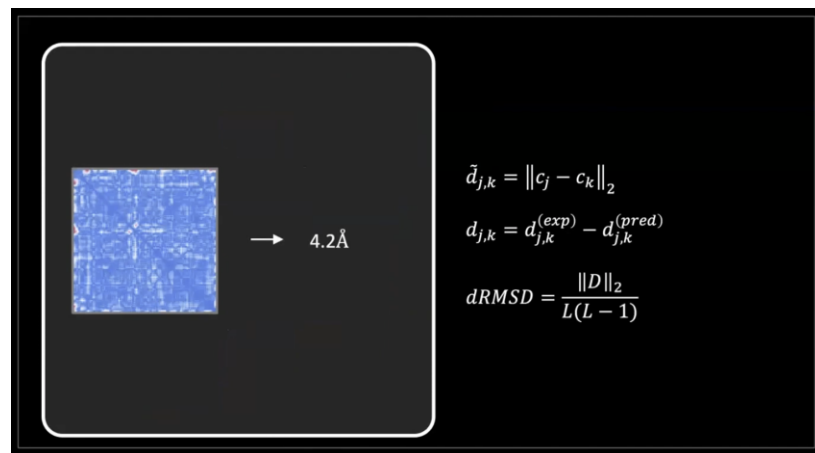


- The same single representation $\mathbf{S}_i$ is used to predict side-chain torsion angles.
- Because **side-chain geometry is largely rigid**, torsion prediction is sufficient.
- The system is trained **end-to-end**.

Structure Quality Can Be Measured Locally and Globally



Local Structure Accuracy (IDDT / LDDT)



dRMSD is a more global distance-based metric, but not the same as GDT (Global Distance Test)

- IDDT (i.e. Local Distance Difference Test) measures local structural accuracy.
- GDT (i.e. Global Distance Test) measures global structural similarity.

AlphaFold 2 Predicts Its Own Confidence via pLDDT

Model Confidence:

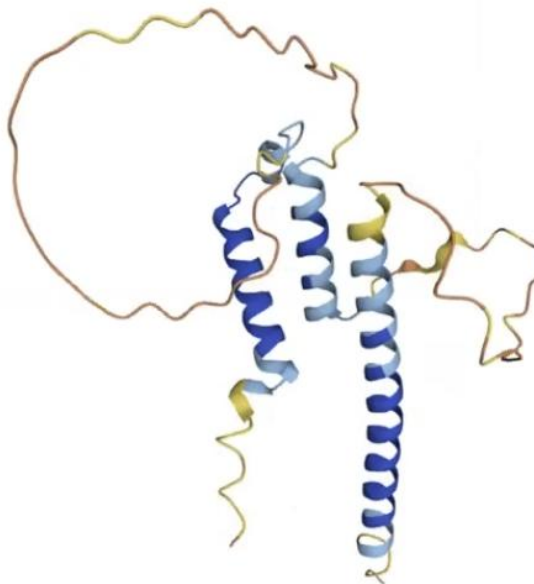
Very high (pLDDT > 90)

Confident (90 > pLDDT > 70)

Low (70 > pLDDT > 50)

Very low (pLDDT < 50)

AlphaFold produces a per-residue confidence score (pLDDT) between 0 and 100. Some regions with low pLDDT may be unstructured in isolation.



- **LDDT** = the **true (local)** evaluation metric
- **pLDDT** = the model's **predicted estimate** of that metric

The First Training Phase Combines Structural and Auxiliary Losses

$$\mathcal{L} = 0.5 \mathcal{L}_{\text{FAPE}} + 0.5 \mathcal{L}_{\text{aux}} + 0.3 \mathcal{L}_{\text{dist}} + 2.0 \mathcal{L}_{\text{MSA}} + 0.01 \mathcal{L}_{\text{conf}}$$

$\mathcal{L}_{\text{FAPE}}$ **FAPE loss** → geometry-aware loss invariant to global rigid motions

$\mathcal{L}_{\text{aux}}$ **Auxiliary loss** → per-block supervision during iterative refinement

$\mathcal{L}_{\text{dist}}$ **Distogram loss** → supervision on the pair representation

$\mathcal{L}_{\text{MSA}}$ **Masked MSA loss** → recovery of masked amino acids from evolutionary context

$\mathcal{L}_{\text{conf}}$ **Confidence loss** → supervision for confidence estimation

The Second Training Phase Adds Experimental and Violation Losses

$$\mathcal{L} = 0.5 \mathcal{L}_{\text{FAPE}} + 0.5 \mathcal{L}_{\text{aux}} + 0.3 \mathcal{L}_{\text{dist}} + 2.0 \mathcal{L}_{\text{MSA}} + 0.01 \mathcal{L}_{\text{conf}} + 0.01 \mathcal{L}_{\text{exp_resolved}} + 1.0 \mathcal{L}_{\text{viol}}$$

Experimentally resolved atom loss ($\mathcal{L}_{\text{exp_resolved}}$):

Not every atom has **experimentally determined coordinates**. AlphaFold 2 includes an auxiliary prediction of **whether each atom was experimentally resolved**. This loss trains that prediction using **resolved/unresolved labels**. Missing coordinates are handled separately: structural losses **ignore comparisons** that require unavailable experimental coordinates.

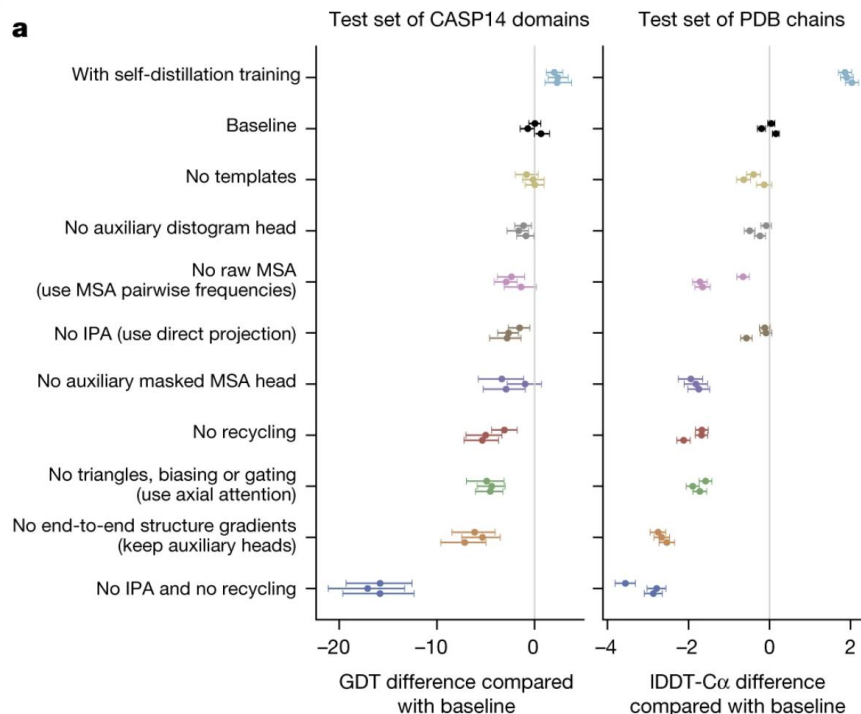
Violation loss ($\mathcal{L}_{\text{viol}}$):

A predicted protein can have the right **overall shape** but still contain **unrealistic local geometry**. This loss penalizes atoms coming too close together (**steric clashes**), incorrect **bond lengths**, and incorrect **bond angles**. During fine-tuning, it encourages **chemically plausible structures**, including in regions **without experimental coordinates**. These are **soft penalties**, so some violations can still remain.

Ablation Studies Reveal Which Components Matter Most

Fig. 4: Interpreting the neural network.

From: [Highly accurate protein structure prediction with AlphaFold](#)



“Highly accurate protein structure prediction with AlphaFold” by J. Jumper et al., Nature, 2021

IPA and Recycling Are Critical for Performance

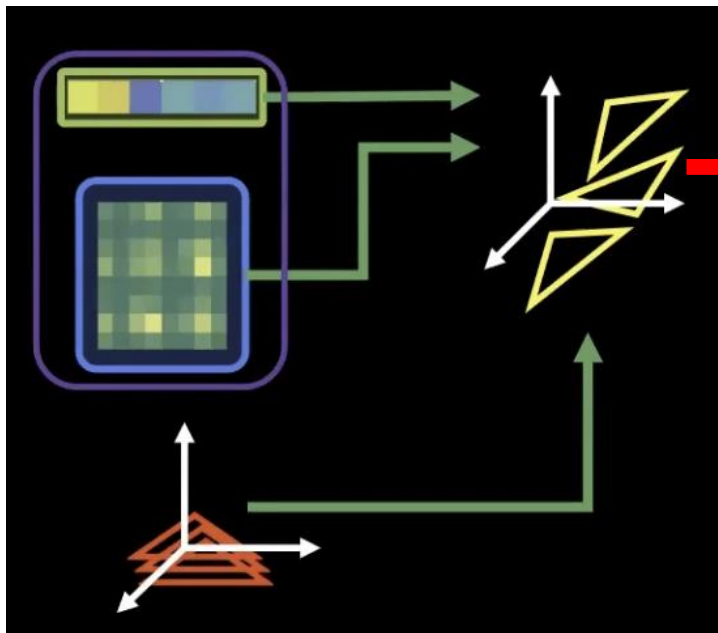
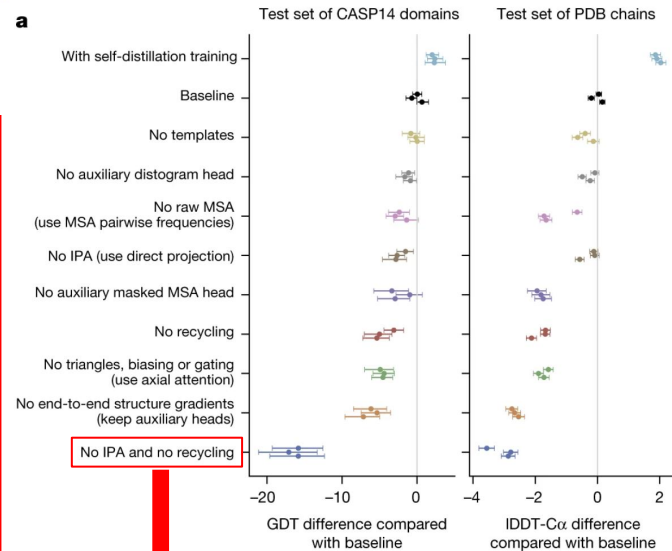


Fig. 4: Interpreting the neural network.

From: [Highly accurate protein structure prediction with AlphaFold](#)



$$a_{ij} = \text{softmax} \left(q_i^T k_j + b_{ij} \times \left\| T_i \circ \vec{q}_i - T_j \circ \vec{k}_j \right\|^2 \right)$$

Remember → In **AlphaFold 2**, **IPA** stands for **Invariant Point Attention**. It's one of the key architectural innovations that allows the model to reason directly in **3D space** while remaining consistent under rotations and translations.

OpenFold

OpenFold Prioritized Reproducibility and Trainability

Models	AlphaFold2	RoseTTAFold	OpenFold
Highly-accurate	✓	✓	✓
Platform	Jax	PyTorch	PyTorch
Data (MSAs)	×	×	✓
Trainable	×	×	✓

GitHub repository interface for **openfold** (Public).

Navigation: Code, Issues (206), Pull requests (24), Actions, Projects, Wiki, Security, Insights.

Repository stats: Watch (45), Fork (660), Starred (3.3k).

Branches: main (7 Branches, 6 Tags).

Recent activity: Merge pull request #560 from borisfom/fix-cueq-scaling (be2ec18 · 2 months ago) by jnwei. 1,612 Commits.

Files: .github (Bump actions/checkout from 4 to 5, 6 months ago), docs (Install docs added, 3 months ago).

About (highlighted): Trainable, memory-efficient, and GPU-friendly PyTorch reproduction of AlphaFold 2.

Tags: protein-structure, pytorch, alphafold2.

Ask DeepWiki for concrete Details

The screenshot shows the DeepWiki interface. At the top, there's a search bar with the query "Where and how is the triangular inequality enforced in a smooth differentiable way?". Below the search bar, the results for the repository "aqlaboratory/openfold" are displayed. The main result is titled "Triangular Inequality Enforcement in OpenFold". It includes a summary paragraph and a detailed table of contents with three sections: "1 Virtual Bond Computation Using Law of Cosines", "2 Distance Bounds Generation for Geometric Constraints", and "3 Residue Type Specific Bond Lengths". The first section is expanded, showing code snippets from "residue_constants.py" that implement the law of cosines to calculate bond lengths. A second section, "2 Distance Bounds Generation for Geometric Constraints", is also visible. On the right side of the interface, there's a code editor showing the full content of "loss.py", which includes imports, license information, and the main loss function. At the bottom, a chat input box contains the question "How is the triangular inequality enforced 'in bulk', i.e. in parallel for efficiency?", with a "Codemap" button and a send icon.

DeepWiki

Index your code with [DevIn](#) [Share](#)

← aqlaboratory/openfold

Where and how is the triangular inequality enforced in a smooth differentiable way?

[Codemap](#) [Open with Windsurf](#)

Triangular Inequality Enforcement in OpenFold

This codemap traces how OpenFold enforces triangular inequality constraints smoothly and differentiably during protein structure prediction. The system computes virtual bonds using the law of cosines [1b], creates distance bounds with tolerance [2c], and enforces them through ReLU-based penalty functions [3a-3c] that are fully differentiable for gradient-based training. Key integration points show how bounds are generated per residue type [4b] and applied in the training pipeline [4c].

1 Virtual Bond Computation Using Law of Cosines

Core geometric constraint computation that translates bond angles into distance bounds. [See guide](#)

Parse bond lengths from file

Parse bond angles from file

Translate angles to virtual bonds

Step	Description	File
1a	Extract bond angle	residue_constants.py:515
1b	Apply law of cosines	residue_constants.py:516
1c	Store virtual bond	residue_constants.py:538

Return bonds, virtual_bonds, angles

2 Distance Bounds Generation for Geometric Constraints

Creating upper/lower bounds from virtual and standard bonds for enforcement. [See guide](#)

make_atom14_dists_bounds() function

Initialize bound arrays

Step	Description
2a	Van der Waals lower bounds
2b	Process all bond types
2c	Process all bond constraints

[Codemap](#)

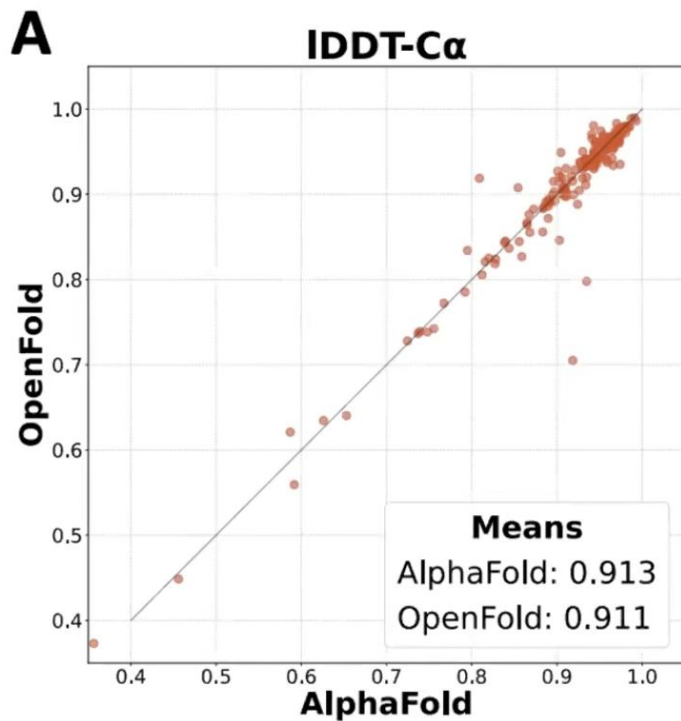
loss.py

```
1 # Copyright 2021 AlQuraishi Laboratory
2 # Copyright 2021 DeepMind Technologies Limited
3
4 # Licensed under the Apache License, Version 2.0 (the "License");
5 # you may not use this file except in compliance with the License.
6 # You may obtain a copy of the License at
7 #
8 #     http://www.apache.org/licenses/LICENSE-2.0
9 #
10 # Unless required by applicable law or agreed to in writing, software
11 # distributed under the License is distributed on an "AS IS" BASIS,
12 # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13 # See the License for the specific language governing permissions and
14 # limitations under the License.
15
16 import al_collections
17 import numpy as np
18 import torch
19 import torch.nn as nn
20 from typing import Dict, Optional, Tuple
21
22 from openfold.np import residue_constants
23 from openfold.utils.rigid_utils import Rotation, Rigid
24 from openfold.utils.geometry.vector import Vec3Array, euclidean_distance
25 from openfold.utils.all_atom_multimer import get_rc_tensor
26 from openfold.utils.tensor_utils import (
27     tree_map,
28     masked_mean,
29     permute_final_dims,
30 )
31 import logging
32 from openfold.utils.tensor_utils import tensor_tree_map
33
34 logger = logging.getLogger(__name__)
35
36
37 def softmax_cross_entropy(logits, labels):
38     loss = -1 * torch.sum(
39         labels * torch.nn.functional.log_softmax(logits, dim=-1),
40         dim=-1,
41     )
42     return loss
43
44
45 def simoid_cross_entropy(logits, labels):
```

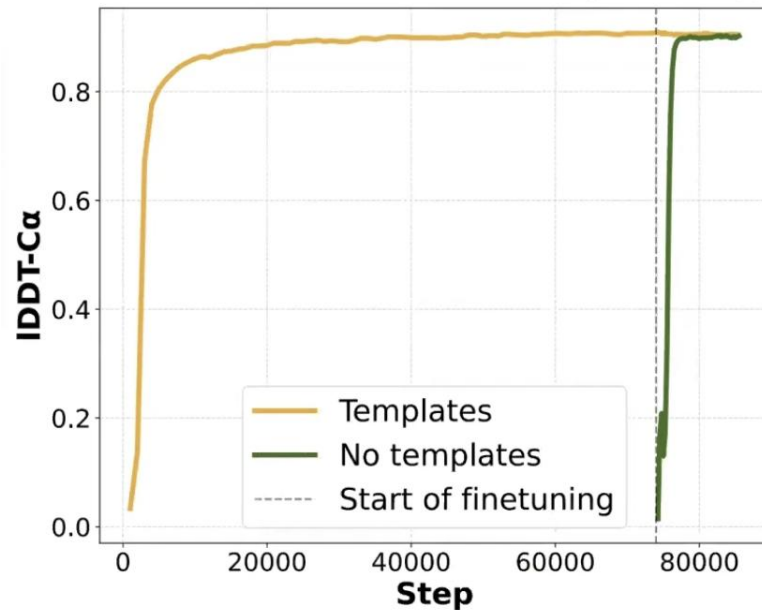
Append github username and repository name to the **deepwiki.com** URL → <https://deepwiki.com/aqlaboratory/openfold>

OpenFold Achieves Near-Parity with AlphaFold 2

CAMEO | Continuous Automated Model EvaluatiOn

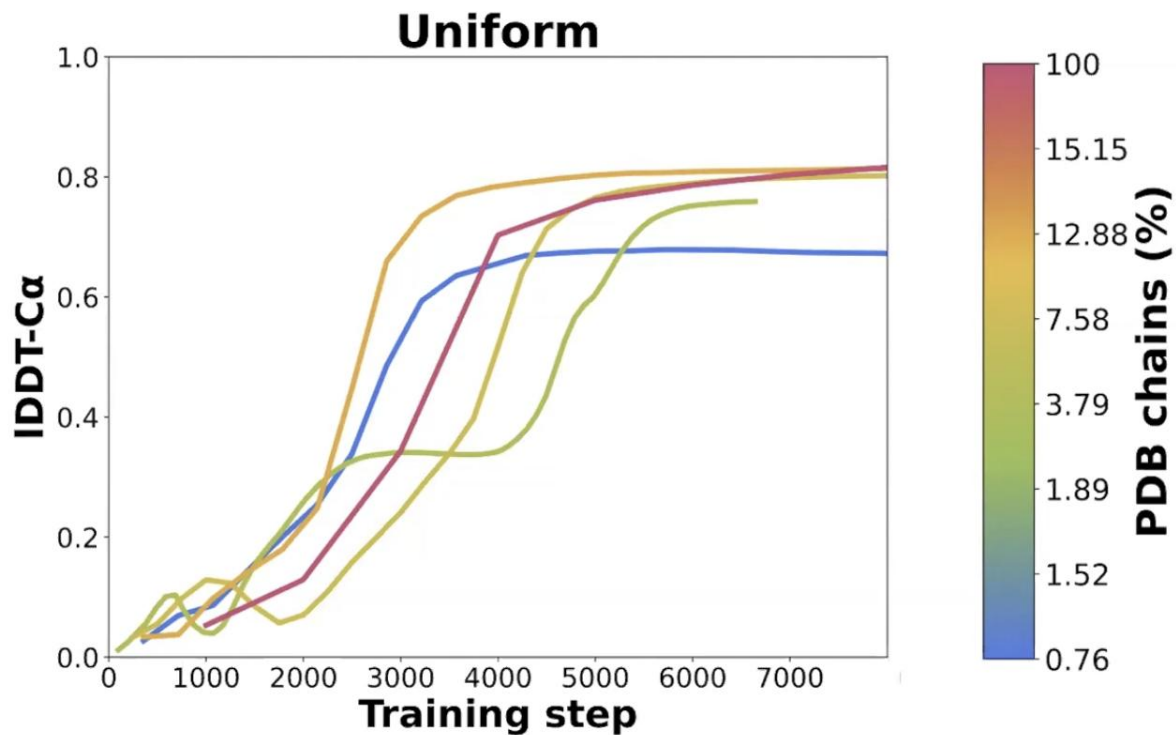


Most Performance Emerges Early in Training

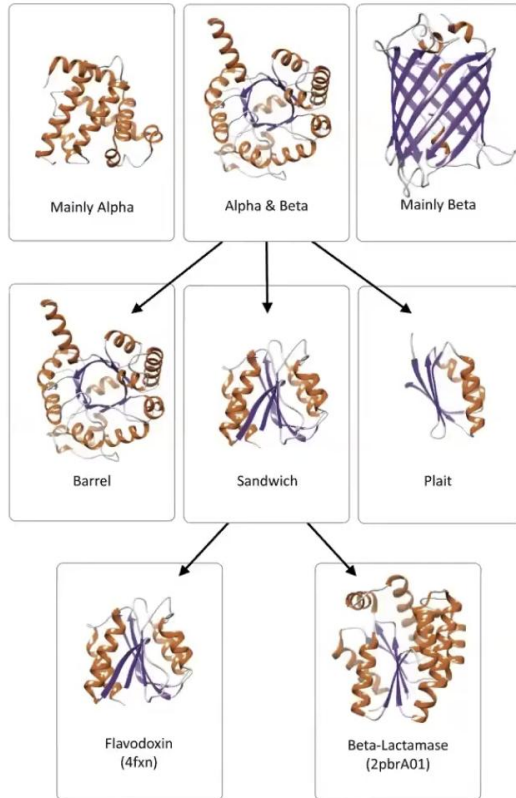


The model achieves **90% of the final metric** in only **3% of the training time**.

Performance with Deliberately Reduced Datasets



Protein Fold Space Is Rich but Still Constrained



Class

Architecture

Topology or Fold

***Nature is
parsimonious in
the structures it
allows***

Take-Home Messages

- AlphaFold 2 turned protein folding into a **learnable geometric reasoning** problem.
- Its success came from combining **co-evolution (1D)**, **pairwise reasoning (2D)**, and **triangle equivariant updates (3D)**.
- The architecture works because its inductive biases **match the biology and the geometry**.
- **Confidence prediction (pLDDT)** and reproducibility made the approach **scientifically useful**.

References

Theory material → <https://nazimbouatta.scholars.harvard.edu/lectures>

“Highly accurate protein structure prediction with AlphaFold” paper, by Jumper et al. → <https://www.nature.com/articles/s41586-021-03819-2>

Nobel Prize Talk → <https://www.youtube.com/live/HnT1VWzdFWc>

CASP challenge blog post → <https://deepmind.google/blog/alphafold-a-solution-to-a-50-year-old-grand-challenge-in-biology/>

OpenFold repository → <https://github.com/aqlaboratory/openfold>

“OpenFold: retraining AlphaFold2 yields new insights into its learning mechanisms and capacity for generalization”, by Ahdriz et al. → <https://www.nature.com/articles/s41592-024-02272-z>

DeepWiki → <https://deepwiki.com/aqlaboratory/openfold>

“MSA Transformer” by Rao et al. → <https://www.biorxiv.org/content/10.1101/2021.02.12.430858v1.full>

“E(n) Equivariant Graph Neural Networks”, by Satorras et al. → <https://arxiv.org/abs/2102.09844>

“Do Transformers Really Perform Bad for Graph Representation?”, by Ying et al. → <https://arxiv.org/abs/2106.05234>

“CCNet: Criss-Cross Attention for Semantic Segmentation”, by Huang et al. → <https://arxiv.org/abs/1811.11721>

“Protein Assembly and Building Blocks: Beyond the Limits of the LEGO Brick Metaphor”, by Y. Levy → <https://www.semanticscholar.org/paper/Protein-Assembly-and-Building-Blocks%3A-Beyond-the-of-Levy/86a997321430edf8815f35d5e7254c1b9e7a1dd2>



Thank You!
Questions?